AI&AC

ASSIGNMENT-21.1

NAME:K.Anjali

HT NO.:2303A51644

BATCH:28

Task-1: AI-Assisted Pair Programming

Prompt:

Create a simple To-Do List Manager in Python. Requirements :Add a task ,View all tasks ,Delete a task ,Use basic lists only (no advanced libraries),Use a simple menu-driven interface ,Keep the code beginner-friendly and minimal

Code:

```python
#Create a simple To-Do List Manager in Python. Requirements:Add a task ,View all tasks,Delete a task
tasks = []
def add_task():
    task = input("Enter a new task: ")
    tasks.append(task)
    print("Task added successfully!")

def view_tasks():
    if not tasks:
        print("No tasks in the list.")
    else:
        print("To-Do List:")
        for idx, task in enumerate(tasks, start=1):
            print(f"{idx}. {task}")

def delete_task():
    view_tasks()
    if not tasks:
        return
    try:
        task_num = int(input("Enter the task number to delete: "))
        if 1 <= task_num <= len(tasks):
            removed_task = tasks.pop(task_num - 1)
            print(f"Task '{removed_task}' deleted successfully!")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")

def main():
    while True:
        print("\nTo-Do List Manager")
        print("1. Add a task")
        print("2. View all tasks")
        print("3. Delete a task")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            add_task()
        elif choice == '2':
            view_tasks()
        elif choice == '3':
            delete_task()
        elif choice == '4':
            print("Exiting the To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

main()
```

s21.py / ...

```python
tasks = []
def display_menu():
    print("\nTo-Do List Manager")
    print("1. Add a task")
    print("2. View all tasks")
    print("3. Delete a task")
    print("4. Mark a task as complete")
    print("5. Save tasks to file")
    print("6. Load tasks from file")
    print("7. Exit")
def add_task():
    task = input("Enter a new task: ")
    tasks.append({"task": task, "completed": False})
    print("Task added successfully!")
def view_tasks():
    if not tasks:
        print("No tasks in the list.")
        return
    for idx, task in enumerate(tasks):
        status = "✓" if task["completed"] else "X"
        print(f"{idx + 1}. [{status}] {task['task']}")
def delete_task():
    view_tasks()
    try:
        task_num = int(input("Enter the task number to delete: "))
        if 1 <= task_num <= len(tasks):
            del tasks[task_num - 1]
            print("Task deleted successfully!")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")
def mark_task_complete():
    view_tasks()
    try:
        task_num = int(input("Enter the task number to mark as complete: "))
        if 1 <= task_num <= len(tasks):
            tasks[task_num - 1]["completed"] = True
            print("Task marked as complete!")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")
def save_tasks_to_file():
    with open("tasks.txt", "w") as file:
        for task in tasks:
            status = "completed" if task["completed"] else "incomplete"
            file.write(f"{task['task']} - {status}\n")
    print("Tasks saved to file successfully!")
def load_tasks_from_file():
    try:
        with open("tasks.txt", "r") as file:
            for line in file:
                task, status = line.strip().split(" - ")
                tasks.append({"task": task
                                , "completed": status == "completed"})
        print("Tasks loaded from file successfully!")
    except FileNotFoundError:
        print("No saved tasks found.")
def main():
    while True:
        display_menu()
        choice = input("Enter your choice: ")
```

ss21.py > ...

```python
#Explain the differences between the original and improved code.
#Refactor this program using functions or object-oriented programming (OOP).Make the code more modular and scalable.
class ToDoList:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append({"task": task, "completed": False})
    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        for idx, task in enumerate(self.tasks, 1):
            status = "✓" if task["completed"] else "✗"
            print(f"{idx}. [{status}] {task['task']}")
    def delete_task(self, index):
        if 0 < index <= len(self.tasks):
            del self.tasks[index - 1]
        else:
            print("Invalid task number.")
    def mark_task_complete(self, index):
        if 0 < index <= len(self.tasks):
            self.tasks[index - 1]["completed"] = True
        else:
            print("Invalid task number.")
def main():
    todo_list = ToDoList()
    while True:
        print("\nTo-Do List Manager")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Delete Task")
        print("4. Mark Task as Complete")
        print("5. Exit")
        choice = input("Choose an option: ")
        if choice == '1':
            task = input("Enter the task: ")
            todo_list.add_task(task)
            print("Task added.")
        elif choice == '2':
            todo_list.view_tasks()
        elif choice == '3':
            index = int(input("Enter the task number to delete: "))
            todo_list.delete_task(index)
        elif choice == '4':
            index = int(input("Enter the task number to mark as complete: "))
            todo_list.mark_task_complete(index)
        elif choice == '5':
            print("Exiting the program.")
            break
        else:
            print("Invalid option. Please try again.")
if __name__ == "__main__":
```

Output:

```
To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: revise the topic
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 1
Enter a new task: revise the topic
Task added successfully!

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 2
To-Do List:
1. Study for 2 hours
2. revise the topic

To-Do List Manager
```

```
ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\la
Enter your choice: 1
Enter a new task: studyy aI
Task added successfully!

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Mark a task as complete
5. Save tasks to file
6. Load tasks from file
7. Exit
Enter your choice: 3
1. [X] studyy aI
Enter the task number to delete: 1
Task deleted successfully!

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Mark a task as complete
5. Save tasks to file
```

```
Enter the task: sleeping
Task added.

To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
4. Mark Task as Complete
5. Exit
Choose an option: 4
5. Exit
Choose an option: 4
Choose an option: 4
Enter the task number to mark as complete: 1

To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
4. Mark Task as Complete
5. Exit
```

Explanation:

1. **The program did not show output because main() was not called.**
2. **Python only runs code that is executed, not just defined.**
3. **All functions were written, but none were started.**
4. **Adding main() at the end starts the program.**
5. **After fixing, the menu appears and the program works properly.**

Task-2:Version Control Integration

Prompt:

```
#Hthe above code is my To-Do List Manager code Improve it by adding error handling, better structure, and new features.Give full updated code.
```

Code:

```
#Hthe above code is my To-Do List Manager code Improve it by adding error handling, better structure, and new features
tasks = []
def add_task():
    task = input("Enter a new task: ")
    if task.strip() == "":
        print("Task cannot be empty. Please try again.")
        return
    tasks.append(task)
    print("Task added successfully!")
def view_tasks():
    if not tasks:
        print("No tasks in the list.")
    else:
        print("To-Do List:")
        for idx, task in enumerate(tasks, start=1):
            print(f"{idx}. {task}")
def delete_task():
    view_tasks()
    if not tasks:
        return
    try:
        task_num = int(input("Enter the task number to delete: "))
        if 1 <= task_num <= len(tasks):
            removed_task = tasks.pop(task_num - 1)
            print(f"Task '{removed_task}' deleted successfully!")
        else:
            print("Invalid task number. Please try again.")
    except ValueError:
        print("Please enter a valid number.")
def main():
    while True:
        print("\nTo-Do List Manager")
        print("1. Add a task")
        print("2. View all tasks")
        print("3. Delete a task")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            add_task()
        elif choice == '2':
            view_tasks()
        elif choice == '3':
            delete_task()
        elif choice == '4':
            print("Exiting the To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

Git commands:

```
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git init
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git init
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git init
Initialized empty Git repository in C:/Users/manis/OneDrive/Documents/AI-Assignents/.git/
Initialized empty Git repository in C:/Users/manis/OneDrive/Documents/AI-Assignents/.git/
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git add .
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git commit -m "Initial commit: basic To-Do List Manager"
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git add .
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git commit -m "Initial commit: basic To-Do List Manager"
 create mode 100644 ass15.py
 create mode 100644 ass21.py
 create mode 100644 queries.sql
 create mode 100644 student.db
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git checkout -b ai-improvements
Switched to a new branch 'ai-improvements'
PS C:\Users\manis\OneDrive\Documents\AI-Assignents>
```

Output:

```
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 1
Enter a new task: Study AI
Task added successfully!

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 2
To-Do List:
1. Study AI

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 3
To-Do List:
1. Study AI
Enter the task number to delete: 1
Task 'Study AI' deleted successfully!

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice:
```

Explanation:

1. **The program is executed using the command python filename.py in the terminal or by clicking the Run button in VS Code.**
2. **After running, the program displays a menu with options like Add, View, Delete, and Exit.**
3. **The program uses input() to take user choices, so it waits for the user to enter a number.**
4. **Based on the user's input, the corresponding function (add, view, or delete) is executed.**
5. **The program continues running in a loop until the user selects the Exit option, after which it stops.**

Task-3: AI for Commit Message Generation Prompt:

```
#Generate a short and meaningful Git commit message for these changes:
#Added input validation and error handling and improve user messages
```

Output:

```
PS C:\Users\manis\OneDrive\Documents\AI-Assignents>
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> git add .
>> git commit -m "Add input validation and error handling; improve user messages"
On branch ai-improvements
nothing to commit, working tree clean
PS C:\Users\manis\OneDrive\Documents\AI-Assignents> python ass21.py

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 1
Enter a new task: listening
Task added successfully!

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 3
To-Do List:
1. listening
Enter the task number to delete: 1
Task 'listening ' deleted successfully!
```

Explanation:

1. **When the program runs, it displays a menu with options like Add, View, Delete, and Exit.**
2. **The program waits for the user to enter a choice using the keyboard.**
3. **Based on the user's input, the corresponding function (add, view, or delete) is executed.**
4. **The program shows the result of the action, such as displaying tasks or confirming deletion.**
5. **The menu repeats continuously until the user selects the Exit option, which stops the program.**

Task-4: Pair Programming Simulation

Prompt:

```
    main()
#this is the above to-do-list manager code Here is my To-Do List Manager code:Improve this program by:
# Adding input validation Adding error handling Improving user interaction
# Provide the improved full code.
```

Code:

```
tasks = []
def add_task():
    task = input("Enter a new task: ")
    if task.strip() == "":
        print("Task cannot be empty. Please try again.")
        return
    tasks.append(task)
    print("Task added successfully!")
def view_tasks():
    if not tasks:
        print("No tasks in the list.")
    else:
        print("To-Do List:")
        for idx, task in enumerate(tasks, start=1):
            print(f"{idx}. {task}")
def delete_task():
    view_tasks()
    if not tasks:
        return
    try:
        task_num = int(input("Enter the task number to delete: "))
        if 1 <= task_num <= len(tasks):
            removed_task = tasks.pop(task_num - 1)
            print(f"Task '{removed_task}' deleted successfully!")
        else:
            print("Invalid task number. Please try again.")
    except ValueError:
        print("Please enter a valid number.")
def main():
    while True:
        print("\nTo-Do List Manager")
        print("1. Add a task")
        print("2. View all tasks")
        print("3. Delete a task")
        print("4. Exit")
        choice = input("Enter your choice: ")
```

```
        choice = input("Enter your choice: ")
        if choice == '1':
            add_task()
        elif choice == '2':
            view_tasks()
        elif choice == '3':
            delete_task()
        elif choice == '4':
            print("Exiting the To-Do List Manager. Goodbye!"
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

Output:

```
To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: Study AI
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: study social
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: sleep
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 2
No tasks in the list.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: ■
```

Explanation:

1. **The program starts by displaying a menu with options like Add, View, Delete, and Exit.**
2. **The user selects an option by entering a number in the terminal.**
3. **Based on the choice, the program performs the action such as adding or deleting a task.**
4. **The program shows the result of the action, like displaying tasks or error messages.**
5. **The menu keeps repeating until the user selects Exit, which stops the program.**

Task: AI-Assisted Debugging Prompt:

```
#the abobe code is my To-Do List Manager code Here is a python program with errors code:Identify all syntax and logical errors.
#Fix the code and provide the corrected version.
#Explain the errors briefly.
```

Error Code:

```
# Provide the improved full code.
tasks = []
def add_task():
    task = input("Enter a new task: ")
    if task.strip() == "":
        print("Task cannot be empty. Please try again.")
        return
    tasks.append(task)
    print("Task added successfully!")
def view_tasks():
    for i in range(len(tasks)):
        print(tasks[i])
add_tasks()
view_tasks()
```

Correct code:

```python
#Corrected version:
tasks = []
def add_task():
    task = input("Enter a new task: ")
    if task.strip() == "":
        print("Task cannot be empty. Please try again.")
        return
    tasks.append(task)
    print("Task added successfully!")
def view_tasks():
    if not tasks:
        print("No tasks in the list.")
    else:
        print("To-Do List:")
        for idx, task in enumerate(tasks, start=1):
            print(f"{idx}. {task}")
def main():
    while True:
        print("\nTo-Do List Manager")
        print("1. Add a task")
        print("2. View all tasks")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            add_task()
        elif choice == '2':
            view_tasks()
        elif choice == '3':
            print("Exiting the To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

Output:

```
To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: Study AI
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: study social
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: sleep
Invalid choice. Please try again.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: 2
No tasks in the list.

To-Do List Manager
1. Add a task
2. View all tasks
3. Delete a task
4. Exit
Enter your choice: ■
```

Explanation:

1. In the corrected version, the function name is changed to `add_task` to reflect its purpose.

2. The `add_task` function is called within a loop in the `main` function, allowing users to add multiple tasks.

3. Additionally, the `view_tasks` function now checks if there are any tasks before trying to print them, providing better user interaction and error handling.

4. The function name `add_tasks` is misleading as it adds only one task. It should be renamed to `add_task`.

5. The `add_tasks` function is called only once, which means only one task

Task 6 – Feature Expansion using AI

Prompt:

```
#Create a simple calculator program in Python.Requirements: Perform basic operations: addition, subtraction, multiplication, and division
# Use a menu-driven interface ,Take input from the user,display the result clearly, Keep the code simple and beginner-friendly
```

Code:

```
# Use a menu-driven interface ,Take input from the user,display the result clearly, Keep t
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b
def main():
    while True:
        print("\nSimple Calculator")
        print("1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")
        print("5. Exit")
        choice = input("Enter your choice: ")
        if choice in ['1', '2', '3', '4']:
            try:
                num1 = float(input("Enter the first number: "))
                num2 = float(input("Enter the second number: "))
            except ValueError:
                print("Invalid input. Please enter numeric values.")
                continue
            if choice == '1':
                result = add(num1, num2)
                print(f"The result of {num1} + {num2} is: {result}")
            elif choice == '2':
                result = subtract(num1, num2)
                print(f"The result of {num1} - {num2} is: {result}")
            elif choice == '3':
                result = multiply(num1, num2)
                print(f"The result of {num1} * {num2} is: {result}")
```

```python
def main():
        if choice == '1':
            result = add(num1, num2)
            print(f"The result of {num1} + {num2} is: {result}")
        elif choice == '2':
            result = subtract(num1, num2)
            print(f"The result of {num1} - {num2} is: {result}")
        elif choice == '3':
            result = multiply(num1, num2)
            print(f"The result of {num1} * {num2} is: {result}")
        elif choice == '4':
            result = divide(num1, num2)
            print(f"The result of {num1} / {num2} is: {result}")
    elif choice == '5':
        print("Exiting the calculator. Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

Output:

```
eDrive\Documents\AI-Assignents\ass21.py
Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 1
Enter the first number: 56
Enter the second number: 87
The result of 56.0 + 87.0 is: 143.0

Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 2
Enter the first number: 43
Enter the second number: 78
The result of 43.0 - 78.0 is: -35.0

Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 4
Enter the first number: 8
Enter the second number: 4
The result of 8.0 / 4.0 is: 2.0

Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
```

Explanation:

1. The improved version of the calculator program includes several new features that enhance its functionality and user experience.

2. Menu-Driven Interface: The improved version provides a clear menu for users to select the desired operation, making it more user-friendly and easier to navigate.

3. Error Handling: The improved version includes error handling for invalid inputs, such as non-numeric values and division by zero. This prevents the program from crashing and provides informative error messages to the user.

4. Clear Output: The improved version displays the results of calculations in a clear and formatted manner, making it easier for users to understand the output.

5. AI helped in improving the application by providing suggestions for better code structure, error handling, and user interface design.
6. It can analyze the code and identify potential issues or areas for improvement, leading to a more robust and user-friendly application.